VIDEO NOTES · PART 6

Advanced Prompting and Output Parsers

Reasoning, reusable templates, and predictable structured data

What this document is

A plain-English guide to three advanced prompting techniques — and to making an LLM return data your code can trust.

1 What This Lecture Covers

The session splits cleanly into two halves. The first improves what goes *into* the model. The second controls what comes *out* of it.

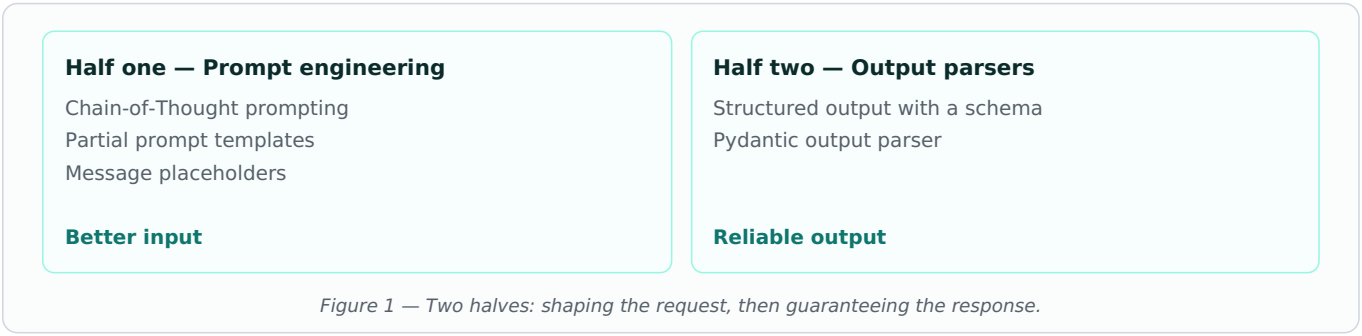

Figure 1 — Two halves: shaping the request, then guaranteeing the response.

Half One — Prompt Engineering
Three techniques for more capable, more reusable prompts

2 Chain-of-Thought Prompting

Ask a model a multi-step question and it will often answer instantly — and confidently wrong. It jumped to a conclusion without working through the parts.

Chain-of-Thought prompting asks the model to lay out its reasoning step by step *before* giving an answer. That intermediate reasoning is not decoration; producing it is what improves the final result.

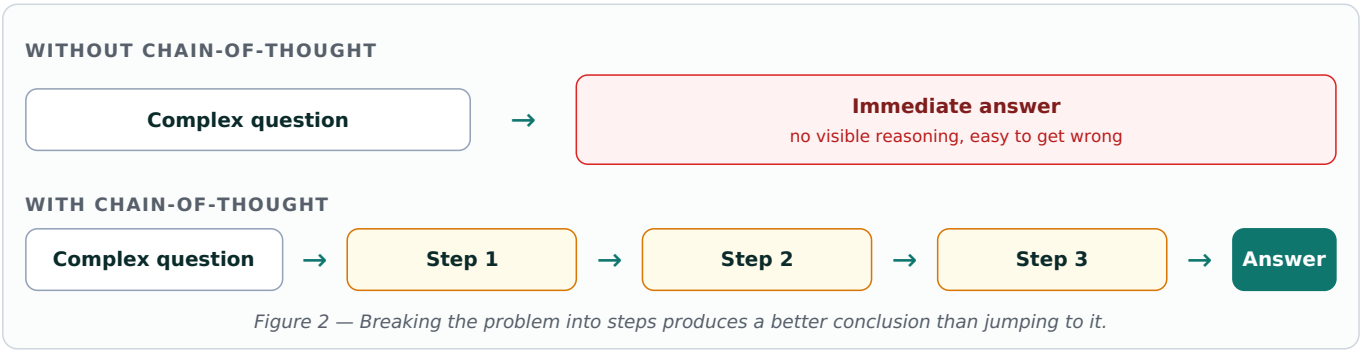

Figure 2 — Breaking the problem into steps produces a better conclusion than jumping to it.

The worked example: an insurance claim

Assessing a claim is exactly the kind of task that fails when rushed. Several conditions must each be checked, and the answer depends on all of them together.

> Assess this insurance claim. Work through it in order:
>
> **1.** Identify what type of claim this is.
> **2.** Check whether the policy covers that type.
> **3.** Confirm the claim falls within the policy dates.
> **4.** Check the amount against the policy limit.
> **5.** Only then, state your decision and the reason.
>
> Claim: `claim_details`

Figure 3 — The prompt itself supplies the reasoning structure, so nothing is skipped.

Why this matters beyond accuracy: the steps are visible. When a decision looks wrong, you can see which step went

wrong — instead of guessing at a single opaque verdict. In a regulated domain, that audit trail may be a requirement, not a nicety.

Chain-of-thought helps	It adds little
Multi-condition decisions, calculations, eligibility checks, diagnosis-style reasoning, anything with dependent steps	Simple lookups, single-label classification, formatting, summarising — tasks with nothing to work out

The cost: reasoning is generated text, so responses are longer, slower and more expensive. Use it where reasoning is genuinely required, not everywhere.

3 Partial Prompt Templates

A normal template waits for every blank to be filled at the moment you use it. But often some values are known long before the rest.

A **partial** template lets you lock in those known values up front, producing a new, smaller template with only the remaining blanks.

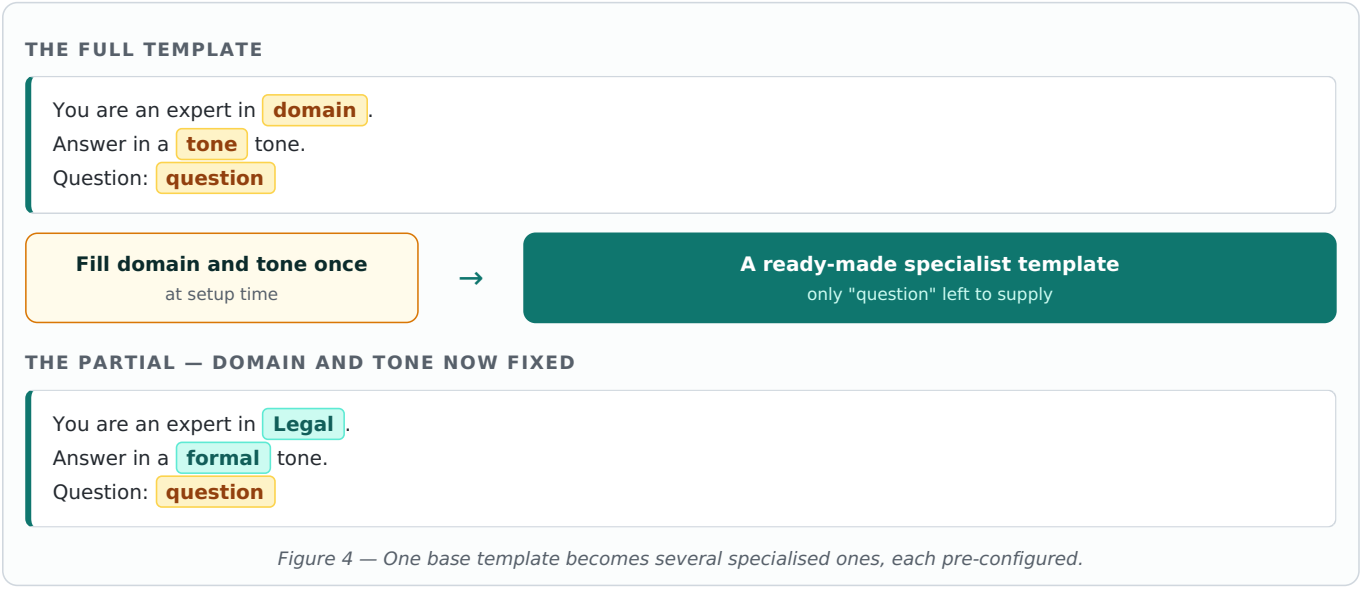

Figure 4 — One base template becomes several specialised ones, each pre-configured.

From a single base template you can produce a legal assistant, an ML/AI assistant and a finance assistant — three configured objects rather than three copies of the same text.

Benefit	What it means in a real project
Modularity	The prompt is authored once; specialisation happens through configuration.
Fewer call sites to change	Values fixed at setup no longer need passing at every single call.
Less room for error	A value locked once cannot be typed differently in one place out of twenty.
Easier improvement	Edit the base wording and every specialised variant improves at once.

> **How to decide what to make partial:** anything that stays the same for the lifetime of a component — domain, tone, output language, the organisation's name — belongs in the partial. What varies per request stays a blank.

4 Message Placeholders

A partial fills a blank with a single value. But sometimes what you need to insert is not one value — it is a whole list of messages whose length you do not know in advance.

MessagesPlaceholder reserves a slot inside a chat template where an entire block of messages can be dropped in at run time.

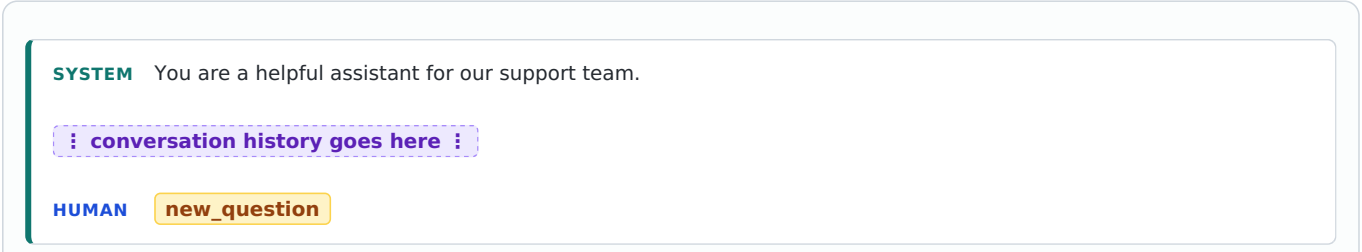

Figure 5 — The placeholder is a reserved gap; its contents are supplied fresh on every call.

Two things commonly go into that gap.

Conversation history

Every earlier exchange in the chat, inserted so the model knows what has already been said.

This is the bridge to the Memory component.

Retrieved documents

Chunks fetched from your own data so the model can answer from them.

This is the bridge to RAG.

Figure 6 — The same mechanism carries both memory and retrieved knowledge into a prompt.

Keep an eye on size. Everything dropped into a placeholder counts toward the model's context limit and toward what you pay. Long conversations and large retrieved chunks both need trimming — which is exactly the problem the memory component handles later in the course.

Half Two — Output Parsers

Turning a text reply into data your code can rely on

5 The Problem With Plain Text

A model answers in prose. Prose is unusable to the code sitting downstream, which needs named fields with predictable types.

Worse, prose is inconsistent. The same prompt run twice may produce the same facts wrapped in different sentences — so any code that tries to pull values out of it breaks the moment the phrasing shifts.

Free text

"Sure! This ticket looks like a billing issue, fairly urgent I would say, and it came from Priya."

Readable. Not usable.

Structured output

category: Billing
priority: High
customer: Priya

Fields your code can read directly.

Figure 7 — The same information, in a form a program can actually use.

6 Structured Output With a Schema

The first approach is to define a **schema** — a description of the fields you want and what each one means. LangChain converts that description into formatting instructions appended to your prompt, then reads the reply back as JSON.

YOU DEFINE THE SHAPE

```
category   text   // which team handles this
priority   text   // Low, Medium or High
customer   text   // name if mentioned
```

Your schema → **Instructions added to the prompt** → **Model** → **JSON matching the schema**

Figure 8 — The schema does double duty: it instructs the model and describes how to read the reply.

> **Write the field descriptions carefully.** They are not comments for your teammates — they are sent to the model and directly shape what it puts in each field. A vague description produces a vague value.

7 The Pydantic Output Parser

Schema-based JSON is a large improvement, but it still has a weak spot: the model returns text that *looks* like JSON, and nothing has checked it properly. A number might arrive as the word "high". A required field might be missing.

Pydantic is a Python library for defining data models with real types. Used as an output parser, it does not just read the reply — it **validates** it.

A PYDANTIC MODEL — TYPES, NOT JUST NAMES

```
category   : str
priority   : Literal["Low", "Medium", "High"]
customer   : str | None
confidence: float
```

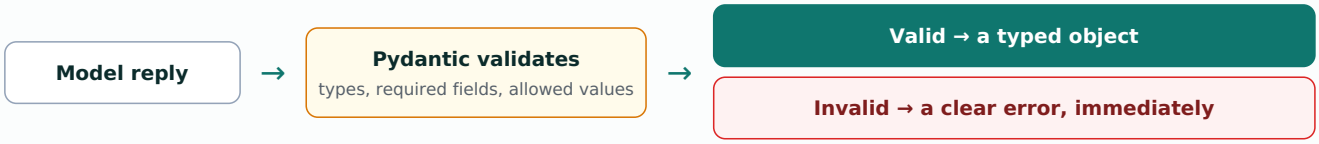

Figure 9 — Validation turns a silent wrong value into a loud, catchable failure.

Why this is described as error-proof

Guarantee	What it prevents
Type enforcement	A field declared as a number cannot arrive as text and quietly break a calculation later.
Required fields	A missing field fails immediately rather than surfacing as an empty value three steps downstream.
Allowed values	A priority field can be restricted to a fixed set, so the model cannot invent "Urgent-ish".
Fails at the boundary	Bad data is caught the moment it enters your system, where the cause is obvious.

> **The honest caveat:** "error-proof" means errors are *caught*, not that they never happen. A model can still return something invalid. The value is that you find out immediately, with a precise message, instead of discovering it in production.

Choosing between the two

Use structured output when	Use Pydantic when
The fields are simple text, the result feeds a human, and a small inconsistency is tolerable	Values feed automated logic, types matter, fields must be constrained, or a wrong value is expensive

8 Putting the Lecture Together

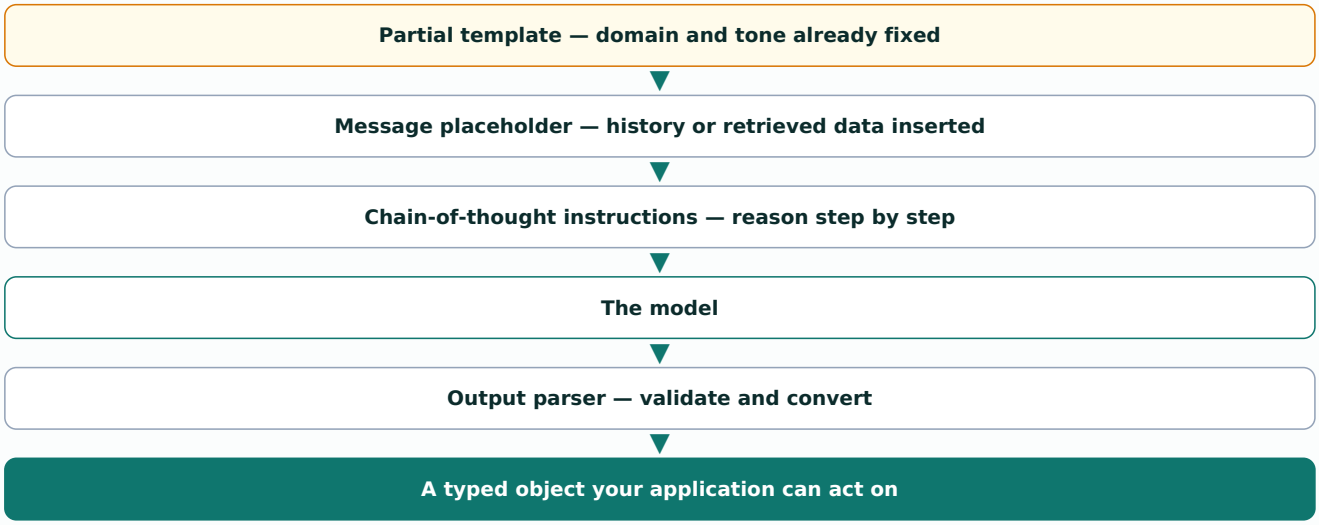

Figure 10 — All five techniques working together in one request.

Technique	Use it when
Chain-of-thought	The task needs working out, not just recall
Partial templates	Some values are fixed for a whole component
Message placeholders	You must insert history or retrieved documents
Structured output	You need named fields instead of prose
Pydantic parser	Those fields must be typed and validated

Practice before the next lecture

- ☐ Take a question the model gets wrong and add explicit reasoning steps to the prompt.
- ☐ Build one base template and derive two specialised partials from it.
- ☐ Add a message placeholder and pass in a short fake conversation history.
- ☐ Define a three-field schema and confirm the reply comes back as clean JSON.
- ☐ Rewrite that schema as a Pydantic model with one restricted-value field.
- ☐ Deliberately make the model break the schema and read the validation error.

> **The short version:** chain-of-thought improves reasoning, partials make templates reusable, placeholders carry history and retrieved data, and output parsers — especially Pydantic ones — turn a text reply into validated data your application can safely act on.